

Container engine

There's one more aspect to the container — the container engine, which accepts users' requests related to the containers. These requests can vary from pulling a container image to running that image. When the container is started, the container engine unfolds all the required files and metadata, and hands it to the Linux kernel for further processing. The container engine makes an API call to the host machine's Linux kernel to start the container process. The latter is just another Linux process running for the host machine. The container image is there before the inspection of the container; this image is turned into the container upon its execution by the container engine. OCI (Open Container Initiative) is a Linux Foundation project which is equipped to design open standards for operating system-level virtualizations; this includes containers. There are several container engines, and some of the popular names are Docker, OpenVZ, CoreOS, and runc.

Networking

Networking within containers has always been an integral part of virtualization. This is one of the reasons why containers were so openly accepted by the software development community. A similar service is provided by Docker. With Docker networking, you can connect either multiple containers to one another or non-Docker workloads to Docker container workloads. Docker manipulates iptables rules on Linux to provide container network isolation.

Network drivers in Docker are responsible for managing the network and providing core network functionalities. There are six network drivers associated with Docker: none, bridge, host, ipvlan, macvlan, and overlay.

None: This disables all the networking within the Docker container.

Bridge: This is the default networking driver provided by Docker to the running containers. With bridged networking, the Docker container can communicate with other running containers connected to the same software bridge.

Host: Containers using host network drivers can use the Docker host networking directly. With this driver, the isolation between the container and the Docker host is removed.

Ipvlan: With Ipvlan, the user gets total control over both IPv4 and IPv6 addressing. It removes the software bridge that traditionally resided between the Docker host NIC and the container interface; instead, the container interface is directly attached to the Docker host interface.

Macvlan: This allows users to set up a MAC address in a container. The container can then appear as a physical device over the network.

Overlay: This allows multiple Docker daemons running over separate host machines to be connected. With the overlay network, users can allow communication between swarm service and standalone containers, or between standalone containers running over different Docker daemons.

Container runtime

Container runtime is another piece of software responsible for running and managing the component that is required to run containers. It can be categorised into two sets — low-level container runtimes and high-level container runtimes.

Low-level container runtime is responsible for just running the containers. This is done by setting up name spaces and cgroups for containers, and there isn't a wide spectrum of features.

High-level container runtime supports more high-level features such as image management, sharing, unpacking, and web APIs or remote procedure calls.

Container runtime also supports the following:

- The image registry is set in the configuration file. From this, the provided registry images are pulled if they are not available locally.
- Images are created as an overlay of various layers. Each layer adds value to the previous existent layer, and together they create a merged file system.
- A container mount point is prepared.
- Metadata is retrieved and applied from the container image. Various settings are configured such as CMD, and ENTRYPOINT.
- The kernel is notified to allocate isolation for processes, networking, and file system to the applicable container (name spaces).
- The kernel is also notified to keep a check on various resource limits like CPU or memory limits to this container (cgroups).
- A system call (syscall) is passed to the kernel to start the container.
- SELinux/AppArmor is set up.

Orchestration

It's easy to manage, say, 10 containers that are running the microservices of an application. You can manually handle the life cycle of these containers. But the task becomes complicated and tedious when we talk about thousands or even hundreds of containers. The manual process of deploying and managing these containers becomes cumbersome. To handle this life cycle of containers and to automate them we use orchestration.

Container orchestration's primary role is to handle containerized applications' deployment, management, scaling, networking, and availability. With container orchestration, enterprises can handle hundreds or even thousands of containers with ease. It can help in